



CGNS Proposal for Extension 0052

Directional Edge-Center GridLocations for 2-D Structured Zones

Draft v0.1 (skeleton — not yet posted)

July 3, 2026

CPEX#	0052 (companion to CPEX 0051)	
Scope	Add <code>IEdgeCenter/JEdgeCenter</code> to <code>GridLocation_t</code> so 2-D structured zones can store directional edge-centered data, symmetric with 3-D's <code>[IJK]FaceCenter</code>	
Champion	M. Scot Breitenfeld	
Organization	The HDF Group	
E-mail	brtnfld@hdfgroup.org	
Origin	Raised during CPEX 0051 pre-submission review	8
GitHub Issue	TBD	
Reference Impl.	TBD	
Target Release	TBD (new enumeration values; additive)	
Date First Posted	TBD (not yet posted; internal draft)	
SIDS Status	skeleton	
Filemap Status	skeleton	
MLL Status	skeleton	

Contents

1 Abstract	3	10
2 Motivation and Rationale	3	11
2.1 Problem Statement	3	12
2.2 Relationship to CPEX 0051	4	13
3 Detailed Description (skeleton)	4	14

3.1	Proposed <code>GridLocation_t</code> Extension	4	15
3.2	<code>DataSetSize[]</code> (sketch)	5	16
3.3	MLL Changes (sketch)	5	17
3.4	Backward Compatibility (sketch)	5	18
4	Design Decisions (skeleton)	5	19
4.1	Convention: Normal-Based, 2-D Only (proposed)	5	20
4.2	Alternative: Full <code>[IJK]EdgeCenter</code> for 3-D As Well (deferred) . . .	6	21
4.3	Alternative: Reuse <code>[IJ]FaceCenter</code> in 2-D (rejected)	6	22
5	Open Questions	6	23
6	References	6	24

1 Abstract

For 3-D structured zones, `GridLocation_t` provides three directional face-center values — `IFaceCenter`/`JFaceCenter`/`KFaceCenter` — because a structured `FlowSolution_t` array is a single `IndexDimension`-dimensional rectangle, and the faces normal to each index direction form differently-shaped rectangles that cannot share one array. The SIDS is explicit that these values are 3-D constructs: “`IFaceCenter` is the center of a face *in 3-D* whose computational normal points in the *i* direction” (Building-Block Structure Definitions, `GridLocation_t`).

2-D structured zones have the same need — the cell interfaces are edges, and the *i*-normal and *j*-normal edge sets are differently-shaped rectangles — but no directional values exist for them. The single undifferentiated `EdgeCenter` value cannot serve: the union of the two edge sets is not one rectangle, so no `DataSet[]` can be defined for it (Section 2). This CPEX adds `IEdgeCenter` and `JEdgeCenter` to `GridLocation_t`, defined for 2-D structured zones by computational normal, in exact analogy with `[IJK]FaceCenter`. The sizing formulas are the 2-D restriction of the existing `[IJK]FaceCenter` formulas; in the MLL, the existing `cgi_datasize()` branch computes them verbatim with `IndexDimension = 2`.

2 Motivation and Rationale

2.1 Problem Statement

A finite-volume code on a 2-D structured grid stores fluxes on cell interfaces: quantities on the edges whose computational normal points in *i*, and separately on the edges whose normal points in *j*. On a zone with `VertexSize [VI, VJ]` and `CellSize [VI-1, VJ-1]`:

- *i*-normal edges form a `[VI, VJ-1]` rectangle;
- *j*-normal edges form a `[VI-1, VJ]` rectangle.

In 3-D the identical situation is served by `IFaceCenter`/`JFaceCenter`/`KFaceCenter`. In 2-D there is no conforming way to store either edge set:

1. **[IJK]FaceCenter cannot be reused.** The SIDS defines them as 3-D constructs (“a face in 3-D”), and the MLL enforces this: `cgi_check_location()` requires `CellDimension ≥ 3` for all three.
2. **Undifferentiated EdgeCenter is structurally insufficient.** It cannot say *which* edge set is stored, and even for “all edges” no `DataSet[]` exists: the two edge sets are differently-shaped rectangles whose union is not a rectangle. This is not an implementation gap — it is the same mathematical fact that forced the directional split for 3-D faces.

3. **The MLL today contains a latent trap here.** `cgi_check_location()` 60
accepts `EdgeCenter` for `CellDimension` ≥ 2 , but `cgi_datasize()` cannot size 61
it (it falls through to “Location not yet supported” for any `CellDimension` > 1). 62
A location that validates but cannot be sized should be reconciled regardless of 63
this CPEX’s fate; to be filed as a CGNS issue in the spirit of [#964/#965](#). 64

The practical workaround today is to model a 2-D problem as a 3-D zone one cell 65
thick, which inflates every array and index computation for no physical reason. 66

2.2 Relationship to CPEX 0051 67

This proposal was raised during CPEX 0051’s pre-submission review. It is kept 68
separate deliberately: CPEX 0051 is scoped to unstructured zones and adds no new 69
enumeration values, while this CPEX is a structured-zone extension whose entire 70
substance is two new enumeration values. Neither depends on the other. 71

3 Detailed Description (skeleton) 72

3.1 Proposed `GridLocation_t` Extension 73

Current text (SIDS, Building-Block Structure Definitions): 74

```
GridLocation_t := Enumeration( 75
    GridLocationNull, GridLocationUserDefined, 76
    Vertex, CellCenter, FaceCenter, 77
    IFaceCenter, JFaceCenter, KFaceCenter, 78
    EdgeCenter ) ; 79
```

Proposed text: 80

```
GridLocation_t := Enumeration( 81
    GridLocationNull, GridLocationUserDefined, 82
    Vertex, CellCenter, FaceCenter, 83
    IFaceCenter, JFaceCenter, KFaceCenter, 84
    EdgeCenter, IEdgeCenter, JEdgeCenter ) ; 85
```

Accompanying definition text (redline sketch, mirroring the existing `IFaceCenter` 86
sentence): 87

```
For 2-D structured zones, IEdgeCenter is the center of an edge 88
whose computational normal points in the i direction; JEdgeCenter 89
is similarly defined. Both are valid only for structured zones 90
with CellDimension = 2. 91
```

3.2 DataSize[] (sketch)

The 2-D restriction of the existing [IJK]FaceCenter rule:

- IEdgeCenter: DataSize[] = [VI, VJ-1] (+ rind, as for IFaceCenter);
- JEdgeCenter: DataSize[] = [VI-1, VJ] (+ rind).

In the MLL, cgi_datasize()'s existing [IJK]FaceCenter branch ("DataSize[j] = dims[j]; decrement every dimension except the matching one") computes exactly these values when run with IndexDimension = 2; the implementation reuses it rather than adding a new formula.

3.3 MLL Changes (sketch)

- cgnslib.h: append IEdgeCenter = 9, JEdgeCenter = 10 to GridLocation_t; bump NofValidGridLocation 9 → 11. Appending preserves the values of all existing members (ABI-additive).
- GridLocationName[] (cgnsKeywords.h and name tables): two new strings.
- cgi_check_location(): new branch — structured only, CellDimension = 2.
- cgi_datasize(): route the new values through the existing directional-face branch.
- cg_sol_write: nothing needed once #964 is fixed by delegation to cgi_check_location (CPEX 0051, MLL Changes); with the allow-list gone, new locations are accepted wherever cgi_check_location says so.
- cgnscheck, Fortran parameter constants, tools: TBD sweep.

3.4 Backward Compatibility (sketch)

New enumeration values are additive: no existing file changes meaning, no existing API input changes behavior. A pre-CPEX reader encountering GridLocation = "IEdgeCenter" sees an unknown enumeration string; version gating (plain CGNSLibraryVersion_t bump, or CPEX-0049 registry entry if adopted) follows the same policy CPEX 0051 adopts.

4 Design Decisions (skeleton)

4.1 Convention: Normal-Based, 2-D Only (proposed)

IEdgeCenter is defined by computational normal, the direct analog of the SIDS's own IFaceCenter definition. In 2-D the in-plane normal of an edge is well-defined,

so the analogy is exact, and the flux-storage use case reads naturally: i-normal fluxes go to `IEdgeCenter`, as i-normal fluxes go to `IFaceCenter` in 3-D.

4.2 Alternative: Full [IJK]EdgeCenter for 3-D As Well (deferred)

3-D structured zones also lack directional edge locations (edge sets along i/j/k are three differently-shaped rectangles). A fuller proposal would add `KEdgeCenter` and define all three for 3-D. Deferred, for one substantive reason: 3-D edges have no computational normal, so the 3-D definition must be *parallel-to-direction* — and an edge *parallel* to j in 2-D is the edge *normal* to i, so a single name cannot carry the normal-based meaning in 2-D and the parallel-based meaning in 3-D without inverting the i/j sense between dimensions. Resolving that naming question (or accepting dimension-dependent semantics) is left open; the 2-D-only scope has no such ambiguity. *[Open question for the committee.]*

4.3 Alternative: Reuse [IJ]FaceCenter in 2-D (rejected)

Relax `cgi_check_location` to accept `IFaceCenter` / `JFaceCenter` for `CellDimension = 2` and redefine them there as edges. Avoids new enumeration values, but contradicts the SIDS’s explicit “a face in 3-D” definition, overloads “face” to mean “edge,” and silently changes what existing readers may assume about the values. Rejected in favor of honest naming.

5 Open Questions

1. Include 3-D [IJK]EdgeCenter now (with the parallel-vs-normal naming question resolved), or keep the 2-D-only scope?
2. Should the latent EdgeCenter validates-but-cannot-size inconsistency be fixed (rejected at validation, or sized for the `CellDimension = 1` case only) as an independent bug, ahead of this CPEX?
3. Version gating: CPEX-0049 registry entry vs. plain `CGNSLibraryVersion_t` bump (same question as CPEX 0051, same answer should apply to both).

6 References

1. CGNS SIDS, Building-Block Structure Definitions, `GridLocation_t` (enumeration and `IFaceCenter` definition text).
2. CGNS SIDS, Structured Grid Notation and Indexing Conventions (cells, faces, edges).

-
3. CPEX 0051, Implicit FaceCenter Indexing for Unstructured Zones with Explicitly Defined Face Sections (origin of this proposal; `cg_sol_write` validation delegation this CPEX relies on). 153
154
155
 4. CGNS/CGNS issues [#964](#), [#965](#) (precedent for filing the **EdgeCenter** check/size inconsistency). 156
157